

AISSD5000 KV Cache 性能测试报告 —— 480B 模型 · TP4×2 双实例

· 长上下文冷恢复

测试平台：AMD Instinct MI308X ×8（每卡 192 GB HBM） / ROCm 7.2 / vLLM 0.20.1+rocm721 + LMCache v1（上游主线源码编译，含磁盘并行读优化）

被测存储：AISSD5000（WS5000）全闪 NVMe-oF 阵列（4 盘 RAID0，RoCEv2 over 100GbE，XFS，14 TB）

对照存储：服务器本地 NVMe（Solidigm，PCIe Gen4，挂载分区 2 TB）；无外存重算

模型：Qwen3-Coder-480B-FP8（MoE，权重约 450 GB）

部署形态：双实例 TP4×2——实例 A 占卡 0-3、实例 B 占卡 4-7，各自独立提供服务（+ expert parallel）

负载：长上下文冷恢复——每会话约 29.8K token，KV $\approx$ 7.15 GB/会话

日期：2026-07-06

一、测试目的与结论摘要

前一轮测试（2026-07-05）已在 8 卡单实例（TP=8）形态下量化了 AISSD5000 对本地 NVMe 与重算的收益。本轮把同一台 8 卡机切换为双实例 TP4×2——即生产上更常见的“一机多服务”切分形态（更高的实例级容错、独立扩缩容、单实例故障不整机停服），回答三个问题：

1. AISSD5000 的收益结论是否跨部署形态成立（而非 TP8 单一形态下的偶然）；
2. 一台 AISSD5000 同时供给多个独立推理实例时，供给能力与稳定性如何；
3. 多实例间经共享阵列复用同一份 KV 的可行性边界在哪里。

主要结论（全部档位逐请求物理冷读取证成立）：

1. 相比本地 NVMe：AISSD5000 首字延迟降低 25% - 30%，聚合输出吞吐提升 28% - 35%，与 TP8 形态的结论（26% - 32% / 29% - 40%）幅度一致——收益跨部署形态稳定成立。全机并发 16 档：TTFT p50 由 19.3 s 降至 13.5 s（降 30%），聚合吞吐由 49.2 提升至 66.5 tok/s（升 35%）。
2. 一套阵列同时喂饱两个独立引擎：双实例并发冷读时阵列峰值 10.33 GB/s（单口 100GbE 线速的 90%+），忙窗均值 8.55 - 8.88 GB/s；本地盘在两种形态下均钉死于 6.78 GB/s 物理上限。灌入阶段双实例并行写 343 GB（0.95 GB/s 持续写 + 双引擎 prefill 读写混合）零异常。
3. 相比无外存重算：TTFT 快 9.5 - 10 倍（p90 口径 16 倍以上）、吞吐高 17 - 20 倍。TP4 实例算力仅为 TP8 的一半，重算 30K 前缀在全机并发 16 时首字要等 2.1 - 4.5 分钟——实例切得越小，重算越不可用，KV 外置层的刚需性越强。
4. 跨实例热共享发现软件边界（诚实负面结果）：实例 B 冷读实例 A 灌入的会话时命中为 0、退化为重算。文件本身在共享阵列上双方都能访问，但 LMCache LocalDiskBackend 的索引是进程内存态、不扫描目录——跨实例复用需要共享索引层（remote/Mooncake 类后端或重启时重建索引）。阵列的容量与带宽已具备条件，缺口在缓存软件而非存储。

二、被测系统与环境

2.1 硬件

组件	配置
GPU	8 × AMD Instinct MI308X (每卡 192 GB HBM, gfx942)
部署形态	实例 A: 卡 0 - 3 (TP=4, 端口 8000); 实例 B: 卡 4 - 7 (TP=4, 端口 8001), 均 + expert parallel
CPU / 内存	2 × AMD EPYC 9654 (384 线程), 约 1.5 TB 内存
被测存储	AISSD5000: 4 盘组 RAID0 (/dev/md0, xfs, 14 TB), NVMe-oF / RoCEv2, 单口 100 GbE
对照存储	本地 NVMe 单盘 (/dev/nvme1n1, PCIe Gen4, 挂 /srv2); 无外存 (重算)

2.2 软件

组件	版本
操作系统	Ubuntu 22.04, 内核 6.8.0-124-generic
GPU 栈	ROCm 7.2 (gfx942)
推理引擎	vLLM 0.20.1+rocm721
KV 缓存库	LMCache (上游主线 2026-06-29 源码编译; 默认异步磁盘加载 + 磁盘并行读优化)
模型	Qwen3-Coder-480B-FP8 (MoE, 权重约 450 GB, 各轮均从 AISSD5000 阵列加载)
关键参数	--tensor-parallel-size 4 --enable-expert-parallel、--max-model-len 32768、--gpu-memory-utilization 0.9、--no-enable-prefix-caching (冷读取证)、LMCACHE_CHUNK_SIZE=256、LMCACHE_MAX_LOCAL_CPU_SIZE=4、PYTHONHASHSEED=0

双实例错峰 30 s 启动; 两实例并行从阵列加载权重 (合计 900 GB), 4.5 分钟双双就绪, 两侧 VRAM 均 90%。

2.3 工作负载与容量基准

- 单会话系统提示前缀约 29.8K token (reps=530), 单会话 KV (TP4 分片合计) 约 7.15 GB;
- 每轮由双实例并行灌入 48 个互异会话 (A 灌 0 - 23、B 灌 24 - 47), 实测落盘 343 GB, 远超显存驻留与 CPU 中转层 (4 GB×2) 容量;

- 测量：两实例同时发起，各读自己灌入的会话（A 读 0..N-1、B 读 24..24+N-1），每会话一次，decode=64，temperature=0；
- 全机并发 = 2 × 每实例并发，档位：8+8=16、16+16=32——与前一轮 TP8 的并发 16/32 档全机口径对齐。

三、测试方法学

3.1 对照设计（三方 × 两档全机并发）

组	KV 后备介质	全机并发	iostat 监控
① AISSD5000	/mnt/ws5000/lmcache480tp4 (md0, RAID0)	16 / 32	/dev/md0
② 本地 NVMe	/srv2/lmcache480tp4_local (nvme1n1, 单盘)	16 / 32	/dev/nvme1n1
③ 重算（无外存）	无（不挂 LMCache）	16 / 32	—（盘读≈0）

三组仅 KV 后备介质不同；双实例布局、模型、权重来源、引擎参数、会话构造、并发档位完全一致。

3.2 物理冷读保证（五重控制）

1. `--no-enable-prefix-caching`：显存不保留任何前缀 KV；
2. 每实例 LMCache CPU 中转层压至 4 GB：内存层无法容纳任何会话；
3. 每档测量前宿主执行 `sync; echo 3 > /proc/sys/vm/drop_caches`：清除 1.5 TB 主机内存页缓存；
4. 每会话仅读取一次，且两实例读取的会话集合不相交（A：0 - 15，B：24 - 39）——不存在跨实例页缓存搭车；
5. 双重取证：四个磁盘组实例日志中全部 96 条测量请求均为 `hit tokens: 30208`（满额磁盘命中）且 `need to load ≈ 30K`；重算组进程无 LMCache 且盘读≈0。

注意：新版 LMCache 默认异步加载，其日志中 `Retrieved ... throughput` 为暂存拷贝速度，不能用于判断物理读盘；物理读一律以 `iostat` 为准。

3.3 测量与采集

- 两实例的测量客户端同时启动（同一秒下发），逐请求测流式 TTFT；
- 聚合输出吞吐 = 全机输出 token 总数（2×N×64）÷ 两实例中较慢者的整批耗时；
- 每档并行采集 `iostat -x 1`（峰值、忙窗均值、忙窗时长）；
- 每组切换彻底清理推理进程（含 EngineCore/Worker 残留）并确认 8 卡显存归零。

四、测试结果

4.1 三方对照总表（TP4×2，双实例同时发起）

档位（全机并发）	TTFT p50 A / B (s)	TTFT p90 A / B (s)	聚合吞吐 (tok/s)	盘峰值	盘忙窗均值	冷读取证
----------	-----------------------	-----------------------	-----------------	-----	-------	------

AISSD5000 • 16 (8+8)	13.17 / 13.89	13.18 / 13.90	66.5	10.33 GB/s	8.55 GB/s (14 s)	16/16
AISSD5000 • 32 (16+16)	26.01 / 27.80	26.01 / 27.81	69.0	10.27 GB/s	8.88 GB/s (27 s)	32/32
本地NVMe • 16 (8+8)	19.34 / 19.29	19.35 / 19.30	49.2	6.78 GB/s	6.31 GB/s (19 s)	16/16
本地NVMe • 32 (16+16)	36.10 / 35.41	36.11 / 35.42	53.8	6.79 GB/s	6.64 GB/s (35 s)	32/32
重算 • 16 (8+8)	132.6 / 125.0	192.6 / 267.9	3.8	≈0	—	无 LMCache
重算 • 32 (16+16)	269.6 / 273.0	435.3 / 466.8	3.4	≈0	—	无 LMCache

4.2 核心对比一：AISSD5000 vs 本地 NVMe

全机并发	本地 TTFT p50	AISSD5000 TTFT p50	TTFT 降低	本地聚合吞吐	AISSD5000 聚合吞吐	吞吐提升
16	19.3 s	13.5 s	-30%	49.2	66.5	+35%
32	35.8 s	26.9 s	-25%	53.8	69.0	+28%

与 TP8 单实例形态（TTFT -26%~-32%、吞吐 +29%~+40%）幅度一致。AISSD5000 的收益不依赖部署形态：无论一机切一个大实例还是两个中实例，本地盘都被 6.78 GB/s 物理顶卡住，AISSD5000 都以 8.6 - 10.3 GB/s 供给。

4.3 核心对比二：AISSD5000 vs 无外存重算

全机并发	指标	重算	AISSD5000	收益
16	TTFT p50（两实例均值）	128.8 s	13.5 s	快 9.5 倍
16	TTFT p90（较差实例）	267.9 s	13.9 s	快 19.3 倍
16	聚合吞吐	3.8 tok/s	66.5 tok/s	高 17.5 倍
32	TTFT p50（两实例均值）	271.3 s	26.9 s	快 10.1 倍
32	聚合吞吐	3.4 tok/s	69.0 tok/s	高 20.3 倍

解读：TP4 实例的 prefill 算力只有 TP8 的一半，重算 30K 前缀在并发下首字要等 2.1 - 4.5 分钟、长尾近 8 分钟——实例切得越小，重算越不可用。生产趋势恰恰是往多实例/小实例切（容错与弹性），这使 KV 外置层从“优化项”变成“部署前提”；在此前提下选介质，由 §4.2 回答。

4.4 部署形态对照：TP8×1 vs TP4×2（同一台机、同一负载规格）

全机并发	形态	AISSD5000 TTFT p50	AISSD5000 吞吐	本地盘 TTFT p50	本地盘吞吐
16	TP8×1（07-05）	11.85 s	74.9 tok/s	17.31 s	53.6
16	TP4×2（本轮）	13.5 s	66.5 tok/s	19.3 s	49.2
32	TP8×1（07-05）	26.35 s	71.6 tok/s	35.73 s	53.9
32	TP4×2（本轮）	26.9 s	69.0 tok/s	35.8 s	53.8

三点观察：

1. 存储供给与形态无关：两种形态下 AISSD5000 峰值均为 10.2 - 10.3 GB/s（单口线速的 90%+）、本地盘均钉死 6.78 GB/s——阵列既能被一个 TP8 实例的 8 路分片并行读打满，也能被两个独立 TP4 实例的并发读打满；
2. 同并发下 TP8 略优、并发 32 时两者持平：TP8 每请求由 8 卡并行读分片并全机 prefill，单请求恢复更快；TP4×2 在并发拉满后（32）与 TP8 几乎重合。形态选择可完全基于业务需要（容错/弹性 vs 单请求延迟），存储不构成约束；
3. 重算的形态敏感性远大于 KV 恢复：KV 恢复从 TP8 换到 TP4×2 只慢 2% - 14%，重算并发 32 档 TTFT 却要 271 s——形态越细分，KV 层价值越大。

4.5 机理验证（带宽账目自治）

- 全机并发 16 需搬运 16 × 7.15 = 114 GB：AISSD5000 按忙窗均值 8.55 GB/s 需约 13.4 s（实测 TTFT p50 13.5 s，吻合）；本地盘按 6.31 GB/s 需约 18.1 s（实测 19.3 s，吻合）；
- 全机并发 32 需搬运 229 GB：AISSD5000 按 8.88 GB/s 需约 25.8 s（实测 26.9 s）；本地盘按 6.64 GB/s 需约 34.5 s（实测 35.8 s）——四个档位的 TTFT 全部由介质带宽解释，测量自治；
- 双实例同时冷读时，两实例 TTFT p50 之差 ≤0.7 s（16 档）/1.8 s（32 档），阵列对两个独立客户端的供给均衡，无饿死/偏斜；
- 灌入阶段双实例并行写：48 会话 / 343 GB / 362 s（≈0.95 GB/s 持续写，与 prefill 读写混合），WS 轮与本地轮灌入耗时几乎相同（363 s vs 361 s）——写入需求远低于两种介质上限，灌入为算力瓶颈，不影响对照公平性。

4.6 机制发现：跨实例热共享需要共享索引层（诚实负面结果）

设计动机：两实例挂同一个阵列目录，理论上 A 灌入的会话 B 应可直接命中（TP4↔TP4 分片格式一致、PYTHONHASHSEED=0 保证 chunk 键一致）。

实测：清页缓存后令实例 B 冷读实例 A 灌入的会话 0 - 3，B 日志显示 4 条请求全部 hit tokens: 0，退化为重算（TTFT p50 67 s）。

根因：LMCache LocalDiskBackend 的键索引在进程内存中维护（写入时登记），不扫描磁盘目录、无跨进程同步——文件虽然就在共享阵列上且两侧都可读，B 的索引里没有 A 写的键，查询直接未命中。

结论与路径：阵列侧的容量、带宽、多客户端并发供给能力都已就位（§ 4.5 第 3 条），跨实例/跨节点共享 KV 池的缺口在缓存软件的索引层——工程上有三条现成路径：① 使用 LMCache remote 后端（集中式索引 + 阵列做数据面）；② Mooncake/Redis 类共享元数据服务；③ 实例启动时目录扫描重建索引（适合“一次预填充、多实例复用”的静态场景）。列为后续验证项。

五、分析与讨论

5.1 收益的稳健性

两轮实验（TP8×1、TP4×2）在同一台机器、同一负载规格下独立完成，AISSD5000 对本地 NVMe 的收益幅度高度一致（TTFT -25%~-32%、吞吐 +28%~+40%）。机理相同：30K 长上下文冷恢复的瞬时带宽需求越过本地盘物理顶（6.78 GB/s），差距即两介质供给能力之差。该结论对部署形态、实例粒度不敏感，可直接外推到“一机 N 实例”的生产切分。

5.2 “一套阵列喂全机”的供给模型

本轮证明单口 100GbE 的 AISSD5000 可同时喂饱两个 480B 实例的恢复风暴（合计 10.3 GB/s 峰值）。按此供给模型外推：增配第二个 100G 口（设备共 6 口）即可支撑 4 实例集群的并发恢复；跨节点场景下多台推理机共享同一阵列，仅受端口聚合带宽约束（满配标称 72 GB/s）。本地盘方案则是每机一套、上限 6.78 GB/s 且容量 2 TB 无法同时容纳权重与 KV 池——多实例化放大而非缩小两者差距。

5.3 KV 层从“优化”变“前提”

TP4 实例重算 30K 前缀的 TTFT（2.1 - 4.5 分钟）比 TP8（2.5 分钟）更差、比 KV 恢复（13.5 - 27 s）差一个数量级。多实例切分是生产常态，而切分越细、单实例算力越小、重算越不可行——KV 外置存储层是多实例部署的前提设施，AISSD5000 在该层的介质对比中稳定胜出。

六、结论

1. 收益跨部署形态成立（核心结论）：480B 双实例 TP4×2 长上下文冷恢复负载下，AISSD5000 相比本地 NVMe 首字延迟降低 25% - 30%、聚合吞吐提升 28% - 35%，与 TP8 单实例轮次（-26%~-32% / +29%~+40%）幅度一致；机理均为本地盘钉死 6.78 GB/s 物理顶、AISSD5000 以 8.6 - 10.3 GB/s 持续供给。
2. 一套阵列同时喂饱两个独立引擎：双实例并发冷读峰值 10.33 GB/s（单口线速 90%+），两实例供给均衡（TTFT 差 ≤1.8 s）；灌入阶段并行写 343 GB 零异常。多实例化放大 AISSD5000 与本地盘的差距。
3. 相比重算快 9.5 - 10 倍（p90 口径 16 - 19 倍）、吞吐高 17 - 20 倍；实例切分越细重算越不可用，KV 外置层是多实例部署的前提设施。
4. 跨实例热共享的边界在软件索引层而非存储：文件级共享已就绪，LMCache LocalDiskBackend 进程内索引导致跨实例不命中；采用集中式索引后端或启动期索引重建即可打通，列为后续验证。

七、局限与后续工作

1. 跨实例共享未打通（§ 4.6）：需以 remote/共享索引后端复测“一次预填充、全机复用”，并进一步外推到跨节点共享；
2. 单机范围：多台推理机共享同一阵列的聚合供给未做物理验证；
3. 重算组档位受时长约束：重算轮完整测了 16/32 两档（合计约 15 分钟），未再扫更高并发；
4. 合成负载：均匀访问、固定长度，相对真实偏斜流量为保守估计；各点为单次测量。

附录 A：复现命令（同事服务器，容器 vllm 内）

A.1 双实例启动（AISSD5000 轮；本地轮把 LMCACHE_LOCAL_DISK 换为

file:///srv2/lmcache480tp4_local，重算轮去掉三个 LMCACHE 变量与 --kv-transfer-config）

```
MODEL=/mnt/ws5000/models/Qwen3-Coder-480B-FP8
for I in 0 1; do
    DEVS=$( [ $I -eq 0 ] && echo "0,1,2,3" || echo "4,5,6,7"); PORT=$((8000+I))
    docker exec -d vllm bash -c "export HIP_VISIBLE_DEVICES=$DEVS VLLM_ROCM_USE_AITER=1 PYTHONHASHSEED=0 \
LMCACHE_CHUNK_SIZE=256 LMCACHE_LOCAL_CPU=True LMCACHE_MAX_LOCAL_CPU_SIZE=4 \
LMCACHE_LOCAL_DISK=file:///mnt/ws5000/lmcache480tp4 LMCACHE_MAX_LOCAL_DISK_SIZE=1000; \
vllm serve $MODEL --served-model-name qwen \
--tensor-parallel-size 4 --enable-expert-parallel --trust-remote-code \
--max-model-len 32768 --gpu-memory-utilization 0.9 --no-enable-prefix-caching \
--kv-transfer-config '{"kv_connector\":\"LMCacheConnectorV1\",\"kv_role\":\"kv_both\"}' \
--port $PORT > /mnt/ws5000/tp4ws_i$I.log 2>&1"
    sleep 30
done
```

A.2 双实例并行灌入（48 会话 / 343 GB）

```
docker exec -d vllm bash -c "python3 /mnt/ws5000/bench_mp.py 8000 populate 530 24 0 > /mnt/ws5000/result
s/tp4_ppA.log 2>&1"
docker exec -d vllm bash -c "python3 /mnt/ws5000/bench_mp.py 8001 populate 530 24 24 > /mnt/ws5000/result
s/tp4_ppB.log 2>&1"
```

A.3 每档测量（示例：全机并发 32 = 16+16；测前必须清页缓存）

```
sync; echo 3 | sudo tee /proc/sys/vm/drop_caches
iostat -x 1 400 /dev/md0 > /tmp/io_tp4_WS_c16x2.log & # 本地轮监控 /dev/nvme1n1
docker exec -d vllm bash -c "python3 /mnt/ws5000/bench_mp.py 8000 measure 530 16 0 64 16 > /mnt/ws5000/re
sults/tp4_WS_c16x2_A.log 2>&1"
docker exec -d vllm bash -c "python3 /mnt/ws5000/bench_mp.py 8001 measure 530 16 24 64 16 > /mnt/ws5000/re
sults/tp4_WS_c16x2_B.log 2>&1"
# 等两个 bench_mp 退出后：
awk ' $1=="md0" { if ($3>m) m=$3 } $1=="md0" && $3>500000 { c++; s+= $3 } END { printf "峰值%.2f GB/s 忙均%.2f GB/s(%d
s)\n", m/1e6, s/c/1e6, c } ' /tmp/io_tp4_WS_c16x2.log
```

A. 4 冷读取证

```
# 每实例 24 条测量请求应全部满额磁盘命中：
grep -acE 'hit tokens: 30[0-9]{3}' /mnt/ws5000/tp4ws_i0.log # 期望 24
grep -acE 'need to load: (2[0-9]{4}|30[0-9]{3})' /mnt/ws5000/tp4ws_i0.log # 期望 24
```

附录 B：负载客户端 bench_mp.py（端口参数化，双实例复用）

```
import urllib.request, json, time, sys
from concurrent.futures import ThreadPoolExecutor
port=sys.argv[1]; mode=sys.argv[2]; reps=int(sys.argv[3]); N=int(sys.argv[4]); off=int(sys.argv[5])
decode=int(sys.argv[6]) if len(sys.argv)>6 else 64
conc=int(sys.argv[7]) if len(sys.argv)>7 else 16
BASE='http://127.0.0.1:%s/v1/chat/completions'%port
basep='背景知识: AISSD5000是国产高性能全闪NVMe-oF存储，可作为大模型推理KV缓存的分层后备介质，配合vLLM与LMCache在显存/内存/磁盘之间分层存取KV。'
def make_prefix(i): return '[sess-%05d] %i + basep*reps
def req(sid, maxtok):
    body=json.dumps({'model':'qwen','stream':True,'messages': [{'role':'system','content':make_prefix(sid)}, {'role':'user','content':'回答%d'%sid}], 'max_tokens':maxtok, 'temperature':0}).encode()
    st=time.time(); ttft=None; n=0
    try:
        r=urllib.request.urlopen(urllib.request.Request(BASE,data=body,headers={'Content-Type':'application/json'}),timeout=1200)
        for line in r:
            sx=line.decode('utf-8','ignore')
            if sx.startswith('data:') and '"content"' in sx:
                if ttft is None: ttft=time.time()-st
                n+=1
            return (ttft, time.time()-st, n)
    except Exception as e:
        return (None,None,0)
def pct(a,q): return a[min(len(a)-1,int(len(a)*q)]) if a else 0.0
if mode=='populate':
    t0=time.time()
    for i in range(N): req(off+i,1)
    print('[p%s] populate N=%d off=%d wall=%.1fs'%(port,N,off,time.time()-t0),flush=True); sys.exit(0)
res=[]; t0=time.time()
with ThreadPoolExecutor(max_workers=conc) as ex:
    futs=[ex.submit(req,off+i,decode) for i in range(N)]
    for f in futs:
        x=f.result()
        if x[0] is not None: res.append(x)
wall=time.time()-t0
tt=sorted(r[0] for r in res); tot=sum(r[2] for r in res)
print('[p%s] n=%d wall=%.1fs TTFT p50=%.3f p90=%.3f mean=%.3f tok/s=%.1f'%(port,len(res),wall,pct(tt,.5),pct(tt,.9),(sum(tt)/len(tt) if tt else 0),tot/wall),flush=True)
```

附录 C：原始数据存档（同事服务器）

文件	内容
/tmp/tp4x2b.out、/tmp/tp4x2c.out	实验全程编排日志（各档 TTFT/吞吐/带宽输出）
/mnt/ws5000/results/tp4_*_{A,B}.log	各档两实例逐档客户端原始输出

/mnt/ws5000/tp4{ws,loc,rc}_i{0,1}.log	六个服务实例完整日志（含逐请求 hit/need 取证行）
/tmp/io_tp4_*.log	各档 iostat 秒级原始记录